

THE CALIBRATION WEIGHTS

mix13 and fmix64: two world-famous finalizers as the known weights of the ladder.

A scale that has never seen a known weight is an assertion. This paper measures the two finalizers the ladder calibrates itself against: the splitmix64 finalizer (Stafford, variant 13) and the MurmurHash3 finalizer (Appleby). Both sit in production a million times over. Both fall on structured counter streams between 16 KB and 2 MB.

Instruments that do not question themselves lie.

0x44 Zero Systems · measured 2026-08-24, revised 2026-08-30

Measurement: RRC smoke, 32 streams per mixer, target 64 MB (2^{26}), PractRand 0.94 (-tf 2)

Raw data: measurement_2026-08-24.json · rig: the ladder

1 WHY CALIBRATION WEIGHTS

The ladder judges other people's mixers. Each of those verdicts is worth exactly as much as the proof that the rig can judge at all — and for that it needs weights whose true behaviour is known. mix13 and fmix64 are those weights.

Everyone knows them. The splitmix64 finalizer sits in Java (SplittableRandom) and in countless seed expanders; the Murmur3 finalizer ships in Murmur3 and in a great many libraries that took it from there. Anyone doubting their fall can re-measure it with the contents of this repository in minutes.

They also play a double role in the instrument. Before any verdict on a stranger the court convenes, and mix13 is the weight that must fall (section 8). In operation their death depths are the yardstick new candidates are read against: the lowest rungs of the ladder.

2 THE TWO SPECIMENS

2.1 mix13 (Stafford, variant 13; the splitmix64 finalizer)

```
x ^= x >> 30; x *= 0xbf58476d1ce4e5b9;  
x ^= x >> 27; x *= 0x94d049bb133111eb;  
x ^= x >> 31;
```

Two multiplications, three xorshift stages. The implementation was verified against the splitmix64 reference on 2026-08-24: the first three outputs for seed 0 (E220A8397B1DCDAF, 6E789E6AA1B965F4, 06C45D188009454F) match bit for bit.

2.2 fmix64 (Appleby; the MurmurHash3 finalizer)

```
x ^= x >> 33; x *= 0xff51afd7ed558ccd;  
x ^= x >> 33; x *= 0xc4ceb9fe1a85ec53;  
x ^= x >> 33;
```

Same construction, different constants and shifts. Verified bit-exact against the original (2026-08-18).

Both are pure multiply-xorshift chains — the construction that was optimised for avalanche metrics and reaches near-perfect scores there. That is their value as calibration weights: they demonstrate that avalanche perfection and survival on the ladder are two different properties.

3 THE SETUP

Per mixer, 32 streams: four disguises (T = identity or bit reversal, C = with or without complement) times eight rotations (r = 0 to 56 in steps of eight). The stream is mixer(ror64(T(counter) ^ C, r)), 64-bit words, judged by PractRand 0.94 (stdin64, -tf 2, -tmin 1KB) up to at most 64 MB = 2²⁶. Test scheme (RRC disguises) after Evensen; judge by Doty-Humphrey.

The verdicts are load-independent: the byte streams are deterministic and PractRand judges content only. No timing numbers appear in this paper.

4 RESULTS

All 64 streams fell. Not one came anywhere near the run target of 2^{26} . Entries: death depth (first FAIL at 2^k bytes).

4.1 mix13 — min 2^{16} , median 2^{19} , max 2^{21}

Disguise	r00	r08	r16	r24	r32	r40	r48	r56
T0C0	2^{19}	2^{18}	2^{17}	2^{16}	2^{19}	2^{19}	2^{17}	2^{16}
T1C0	2^{16}	2^{20}	2^{20}	2^{20}	2^{18}	2^{20}	2^{21}	2^{19}
T0C1	2^{19}	2^{18}	2^{17}	2^{16}	2^{19}	2^{19}	2^{16}	2^{16}
T1C1	2^{16}	2^{20}	2^{20}	2^{20}	2^{18}	2^{20}	2^{21}	2^{19}

4.2 fmix64 — min 2^{14} , median 2^{16} , max 2^{18}

Disguise	r00	r08	r16	r24	r32	r40	r48	r56
T0C0	2^{17}	2^{16}	2^{14}	2^{16}	2^{16}	2^{15}	2^{14}	2^{15}
T1C0	2^{15}	2^{16}	2^{17}	2^{18}	2^{15}	2^{17}	2^{17}	2^{18}
T0C1	2^{18}	2^{17}	2^{14}	2^{15}	2^{15}	2^{15}	2^{15}	2^{15}
T1C1	2^{15}	2^{16}	2^{17}	2^{18}	2^{15}	2^{17}	2^{17}	2^{18}

4.3 Causes of death (last FAIL test per stream)

Test	mix13	fmix64
[Low8/32] Gap-16	15	23
[Low4/32] Gap-16	9	9
BRank(12), incl. Low4/Low8	8	0
Total	32	32

For fmix64 all 32 death verdicts come from Gap-16 on the low bits; for mix13 it is 24 of 32, the rest binary rank. These counts are read from the run logs, not from the JSON, because the raw data records depths and not test names.

5 FINDINGS

5.1 The more famous one is the weaker one

fmix64 dies earlier than mix13: median 2^{16} against 2^{19} , maximum 2^{18} against 2^{21} . The finalizer of Murmur3 sits roughly an order of magnitude below the splitmix64 finalizer on counter streams, at identical construction and identical cost.

5.2 The complement shifts at most one checkpoint here

At T0 the C0 and C1 rows differ by at most one doubling (mix13: one cell, fmix64: five). At T1 they agree cell for cell in all eight rotations, for both mixers. The obvious suspicion, that the C flag does not apply under bit reversal, was tested after exactly that objection: 4 MB of each of the four streams hashed, all hashes distinct. Again for both mixers, and likewise for the diploma feeder. The streams ARE different; their death depths coincide at checkpoint granularity under T1. I have no structural explanation for that — it stands here as an open observation.

What remains is the family contrast: in the specious family (see *The Specious Family*) the complement decided between life and death; all 260 failures sat on streams without complement. Here it shifts at most one checkpoint. The disguise still separates constructions, but “insensitive up to one checkpoint” is the measurable statement; “transparent” would be a claim about mechanism that this measurement does not carry.

5.3 The weakness lives in the low bits

56 of 64 death verdicts come from tests on the low bits (Low4/Low8 foldings), and the bit-reversal streams of mix13 outlive the identity streams (max 2^{21} against 2^{19}). Both fit the same picture: the low-order output bits of these chains carry the strongest residual structure. That is the same place where lu9 later tore (FPF on the lowest bits, see *Where K9 Ends*).

5.4 Position on the ladder

Both calibration weights die 17 to 24 doublings below the depth at which lu9 fell (2^{38}), and 19 to 26 below the diploma target (2^{40}). The distance between the lowest rungs and diploma height is the instrument's working range.

6 COMPARISON WITH EVENSEN

Evensen measured both finalizers under his procedure RR-64-40-TF2-0.94 and published the ranges: 64 rotations times identity/bit-reversal = 128 substreams, no complement, target 1 TB, PractRand 0.94 with -tf 2. His numbers: Murmur3 finalizer 2^{14} to 2^{18} , variant 13 2^{16} to 2^{22} .

This measurement (8 of the 64 rotations, but with complement added, target 64 MB): fmix64 2^{14} to 2^{18} , identical. mix13 2^{16} to 2^{21} , same minimum. That his maximum sits one checkpoint above mine is expected: his 64 rotations are a superset of my eight, and a maximum over more streams can only be greater or equal.

Two independent implementations and two independent rigs arrive at the same ranges. That is the strongest evidence available that the ladder measures what it is supposed to measure. Source: mostlymangling.blogspot.com, “Better, stronger mixer and a test procedure” (January 2019).

7 THE ANCHOR THAT ERASED ITS OWN EVIDENCE

The worst thing an anchor has done on this rig happened between 21 and 26 August 2026, and it went unnoticed for five days.

The diploma runner checks its candidate on four gauntlet rotations before every start — the four historically deadliest disguises, at 256 MB. Those four rotations are also four of the 256 diploma streams. Same mixer, same rotations, therefore the same filename, opened in truncating mode:

```
out = open(OUT / f"{label}.txt", "wb")
```

So every start and every resume overwrote the 1 TB logs of those four streams with a 256 MB anchor result. Resume then skipped them, because the checkpoint had already booked them. The evidence was destroyed and never regenerated, and it hit precisely the four rotations that had been picked for being the most dangerous.

What survived, and what did not.

The verdicts themselves were real. The function that produces them reads nothing but file content and can only return “clean to 2⁴⁰” if a 2⁴⁰ block stood in the file. Run on the same four labels today it returns 2²⁸ — so the files changed after they were read. Their recorded elapsed times place them within seconds of six streams whose complete 1 TB logs are still on disk. The reconstruction was solid.

It was also not good enough. A result that can only be believed through a forensic argument is not a publishable result. The four streams were re-measured from scratch on 28 August, after the overwrite was fixed: all four came back clean to 2⁴⁰, identical to what the checkpoint had recorded seven days earlier. All 256 raw logs are now complete.

The rule that came out of it.

A check must not touch the evidence it checks. The anchor now writes into its own namespace, and the fix was proven both ways: with it the file's timestamp is unchanged to the nanosecond; without it the same call truncates the file from 2796 to 1633 bytes. The second half is the part that matters: a fix nobody has watched fail in its absence has not been proven.

8 SIX ANCHORS INSTEAD OF TWO

Until 28 August the court convened with two weights: mix13 must fall, a known-strong mixer must hold. They prove the rig judges *correctly*, and they quietly assume that it judged at all. A mixer that has not yet fallen after two kilobytes violates no expectation. Both anchors would have waved through the silent exit of PractRand 0.94 — the missing return value that let every mixer look clean to 2¹⁰.

An audit found the same gap one level higher, in the layer that reads a verdict rather than the one that writes it: ten separate places asked “is there a FAIL?” and treated the absence of one as a pass. That is only equivalent if the verdict space has two values. It has four, and two of them carry no FAIL.

Name	Verdict	Anchor	What it proves	Cost
THE GUILTY	FAIL	mix13 must fall	the rig can convict	minutes
THE INNOCENT	clean	NASAM must hold (Evensen)	it does not convict everyone	minutes
THE MISTRIAL	NOT MEASURED	an unknown mixer name	it notices that nothing was measured	0 s

Name	Verdict	Anchor	What it proves	Cost
THE ADJOURNED	ABORTED	a watchdog kill	it notices that it stopped early	3 s
THE SLEEPER	(it bites)	a feeder that writes 16 KB and freezes	the stall detector fires at all	11 s
THE PATIENT	(the margin)	a healthy stream	and it does not cut off the living	2 s

The two new ones each check two things: that the right word comes back, and that the layer reading it refuses to count it as clean. The bug never lived where the verdict is written.

Their triggers are deliberately mundane. The mistrial is a mixer name that does not exist (a typo, the most realistic failure there is); the feeder dies, PractRand receives nothing, and the old code called that clean. The adjournment is a real stream with the watchdog set to three seconds. Together they cost four and a half seconds, which is why they now run in front of the expensive pair rather than after it: if the court cannot say “I did not rule on this”, nothing it says afterwards is worth the minutes.

8.1 AN ANCHOR YOU MEASURED YOURSELF IS NOT A WEIGHT

The upper weight in the table above used to read *a known-strong mixer must hold*, and in the six commands that do the actual measuring that sentence was satisfied by mfx9, a mixer this project found itself. It reads as an anchor and behaves like one: it holds the gauntlets and it fails loudly when the rig breaks, both for the price of seconds. But nobody outside this rig has ever measured it. “mfx9 holds” therefore proves only that the rig still agrees with yesterday’s rig — a consistency check wearing the clothes of a calibration.

A weight is a weight because somebody else already weighed it. mix13 qualifies: Evensen publishes 2^{16} – 2^{22} , this rig measures 2^{16} – 2^{21} , and the overlap is the calibration. On 29 August the upper slot was refilled on the same grounds, with NASAM instead. Evensen publishes it as passing RRC-64-42, and it holds all four gauntlets to 2^{28} in eighteen seconds. Anyone can re-measure that this afternoon: same cost as before, and now checkable from outside.

What the slot had been hiding is a second, different question. Whether the candidate about to be measured already dies at 256 MB is very much worth knowing before spending a week on it — but that is a pre-flight check on the candidate, not a weight. It has its own slot now, and the calibration pair no longer changes from one run to the next.

What makes a mixer an anchor is that somebody else published its range. Trust has nothing to do with it: agreeing with an anchor has to mean something other than agreeing with yourself.

8.2 A DETECTOR BUILT FOR THE WRONG SHAPE OF ITS OWN FAILURE

The rig carries a stall detector, added after the nights of 17./18. August 2026, when roughly every tenth feeder and PractRand pair froze and held a whole run hostage. It was given an anchor at once (a healthy stream must survive it), and that anchor passed every time it ran.

It was the wrong half. A margin test can only show that a detector *saves* the healthy. Whether it ever bites was never asked in front of a measurement, and the answer, for the detector’s entire existence, was no. It checked whether a stream had produced a *first* checkpoint. The stall it is named after does not happen before the first checkpoint: it happens at about 8 KB, four checkpoints in. The comment directly above the check said so, in the same sentence in which the code assumed the opposite.

It surfaced the way these things do — as a number that made no sense. A cost-class run was managing a fraction of the throughput the machine had just been measured to deliver. Two explanations were measured first and both were wrong: the generic chain interpreter is not slower than a compiled mixer (28.1 s against 27.6 s for the same 256 MB), and the watchdog is not counting from the start of a batch. Then one stalled stream's log answered it in four lines: 8 kilobytes at 0.6 seconds, then nothing at all, then the timeout 119 seconds later. Sixty-four streams like it in that batch, killed twice each, paying four minutes each to produce nothing.

How common is that? Counted afterwards from the 92,091 stream logs the finished cost-5 campaign wrote: **1,337 carry the stall abort (1.5 %) and 72 the timeout (0.1 %)**. The cumulative share is the honest one, because the stalls are not spread evenly — the watcher counted hundreds of them in the first ten-minute window of the run and none at all in most later ones. A rate read off one window would have been wrong by an order of magnitude in either direction, depending on which window. Those 92,091 logs are published as an archive with release v1.0, so both shares can be recounted.

The repair is one line of arithmetic and one new anchor. A checkpoint is written every time the data volume doubles, so the wait for the next one is about as long as everything before it; four times that, floored at a constant, is generous at every depth at once — twenty seconds at 8 KB when this was written, most of a day at half a terabyte. And the second half of the pair now exists: a feeder that writes 16 KB of real bytes and then freezes, reproducing the failure exactly. It comes back aborted, and it runs in front of every measurement.

Addendum of 30 August 2026: the floor was measured rather than assumed and dropped from twenty seconds to eight. Eight healthy 256 MB streams were polled ten times a second under a running campaign; the longest gap any of them left between writes sets the floor, and the anchor above now comes back aborted in about ten seconds. The arithmetic is unchanged, and only the constant under it is now a measurement.

Every anchor in this report has a twin now, and this is why. A check that has only ever been seen to pass has not been seen to work; the direction nobody tests is the direction that quietly stops working.

9 WHAT THIS DOES NOT SAY

This paper is not a critique of mix13 or fmix64. Both were designed as hash finalizers, optimised for avalanche, and serve that purpose excellently; the ladder changes nothing about that. What is measured here is a different property: how long a mixer keeps structured counter streams indistinguishable from randomness. A tool fails on the ladder because it was never built for this task, not because it is bad. Which is why the two make honest weights: their fall reproduces, and it was both expected and understood.

Attribution: mix13 by David Stafford; fmix64 by Austin Appleby; splitmix64 reference by Sebastiano Vigna; RRC disguises by Pelle Evensen; PractRand by Chris Doty-Humphrey.

Rig and method: see *The Ladder*. Cost bounds: see *The Cost Limit*. Specious family and the complement effect: see *The Specious Family*. The tear at 2^{38} : see *Where K9 Ends*.